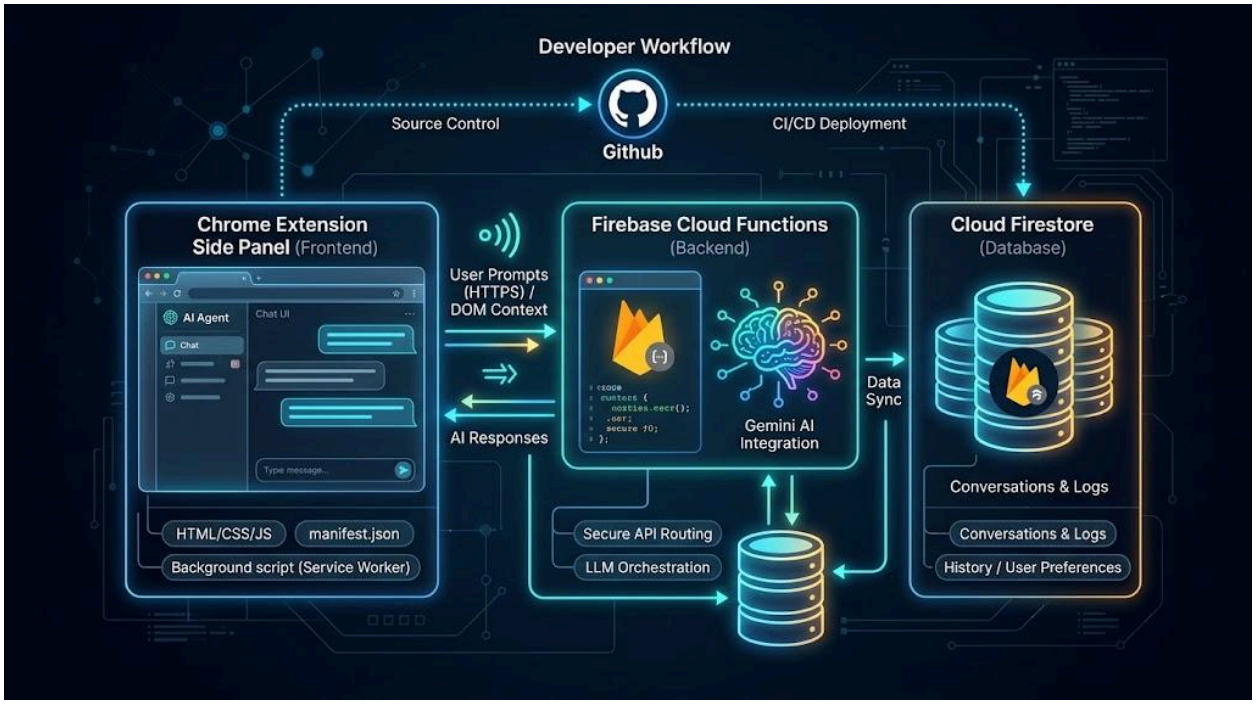
Building AI Agent Chrome Extensions for Automation

Developing a Chrome extension powered by an autonomous AI agent requires a secure, scalable, and modern architecture. Because modern Large Language Models (LLMs) demand secure handling of API credentials and heavy compute logic, client-side execution alone is insufficient.

By pairing **GitHub (for code management and CI/CD pipelines) with **Firebase** (for backend hosting, serverless functions, database storage, and authentication), you can build a production-ready, enterprise-grade AI assistant.**

This comprehensive guide walks through building a secure, 3-layer AI agent extension.



The 3-Layer System Architecture

A robust AI browser extension splits responsibilities across three distinct tiers:

1. **Frontend (Chrome Extension / Manifest V3): Interfaces directly with the user via a side panel and utilizes Chrome APIs (`chrome.sidePanel`, `chrome.scripting`, `chrome.tabs`) to read DOM states and manipulate browser elements.**
2. **Backend (Firebase Cloud Functions & Firestore): Acts as the secure proxy. It isolates your LLM API keys from the client, handles business logic, processes web context, and logs state history.**

3. **Version Control & CI/CD (GitHub & GitHub Actions):** Manages code revisions, tracks prompt templates/agent capabilities, and automates zero-downtime backend deployments.

```
[ Chrome Extension (Side Panel / Content Script) ]
                  │
                  │ (Passes DOM context & user prompts securely via HTTPS)
                  ▼
     [ Firebase Cloud Functions ] ──────► (Calls Gemini / OpenAI API safely)
                  │
                  │ (Saves session history & usage logs)
                  ▼
  [ Cloud Firestore / Firebase Auth ]
```

> **Security Note:** Never embed raw API secrets (like Google Gemini or OpenAI keys) inside client-side Chrome extension scripts. Because extensions can be easily unpacked by users, hardcoded keys will be compromised. Always proxy requests through a secure serverless backend.

Step 1: Setting Up the Chrome Extension Frontend

Create a directory for your extension and configure a Manifest V3 compliant setup.

`manifest.json`
```json
{
  "manifest_version": 3,
  "name": "Enterprise AI Agent",
  "version": "1.0.0",
  "permissions": [
    "activeTab",
    "scripting",
    "sidePanel",
    "storage"
  ],
  "host_permissions": ["https://*/*"],
  "background": {
    "service_worker": "background.js"
  },
```

```
  "side_panel": {
    "default_path": "sidepanel.html"
  },
  "action": {
    "default_title": "Open AI Agent"
  }
}
```

`background.js`
This service worker handles extension icon interactions and acts as a bridge for executing DOM-level automation commands.

```javascript
// Open the side panel natively when the browser extension icon is clicked
chrome.action.onClicked.addListener((tab) => {
  chrome.sidePanel.open({ windowId: tab.windowId });
});

// Message routing listener for executing autonomous browser tasks
chrome.runtime.onMessage.addListener((message, sender, sendResponse) => {
  if (message.type === "EXECUTE_DOM_ACTION") {
    chrome.scripting.executeScript({
      target: { tabId: message.tabId },
      func: executeAgentTask,
      args: [message.action]
    });
  }
});

function executeAgentTask(action) {
  const element = document.querySelector(action.selector);
  if (!element) return;

  if (action.type === "CLICK") {
    element.click();
  } else if (action.type === "TYPE") {
    element.value = action.text;
    element.dispatchEvent(new Event('input', { bubbles: true }));
  }
}
```

Step 2: Provisioning the Firebase Backend

Initialize your Firebase project locally to manage serverless infrastructure.

```bash
npm install -g firebase-tools
firebase login
firebase init functions firestore auth
```

Writing the Agent Controller Cloud Function
Using Node.js and Firebase Cloud Functions (v2), write an HTTP endpoint that ingests page elements, pairs them with system instructions, and queries the LLM to output structured execution steps.

`functions/index.js`
```javascript
const { onRequest } = require("firebase-functions/v2/https");
const { initializeApp } = require("firebase-admin/app");
const { GoogleGenerativeAI } = require("@google/generative-ai");

initializeApp();
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

exports.runAgentStep = onRequest({ cors: true }, async (req, res) => {
** try {**
**   const { prompt, domTree } = req.body;**

**   if (!prompt) {**
**   return res.status(400).json({ error: "Missing prompt parameter." });**
**   }**

**   const model = genAI.getGenerativeModel({**
**   model: "gemini-2.5-flash",**
**   systemInstruction: `You are an autonomous web automation agent.**
**   Analyze the provided simplified DOM tree and user goal. Return a clean JSON object with:**
**   - "thought": string detailing your tactical reasoning**
**   - "action": {"type": "CLICK" | "TYPE", "selector": "CSS_SELECTOR", "text": "optional string if typing"}`**
**   });**

```
    const agentInput = `Page context: ${JSON.stringify(domTree)}\nUser task: ${prompt}`;
    const result = await model.generateContent(agentInput);

    return res.status(200).json({ response: result.response.text() });
  } catch (error) {
    console.error("Agent execution error:", error);
    return res.status(500).json({ error: "Internal server error executing agent." });
  }
});
```

Make sure to securely configure your Gemini secret key within Firebase environment configuration:

```bash
firebase functions:secrets:set GEMINI_API_KEY
```

Step 3: Connecting the Extension to Firebase

Inside your extension's UI workflow (`sidepanel.js` linked from `sidepanel.html`), capture page elements dynamically and pass them to your cloud function endpoint.

```javascript
document.getElementById("send-btn").addEventListener("click", async () => {
  const [tab] = await chrome.tabs.query({ active: true, currentWindow: true });

  // 1. Scrape structured, interactive DOM components safely
  const [{ result: domData }] = await chrome.scripting.executeScript({
    target: { tabId: tab.id },
    func: () => {
      return Array.from(document.querySelectorAll("button, a, input, select")).map(el => ({
        tag: el.tagName.toLowerCase(),
        id: el.id,
        className: el.className,
        text: el.innerText?.trim() || el.placeholder || ""
      }));
    }
  });
```

```
  // 2. Transmit payload to Firebase Backend
  const apiEndpoint =
"https://<YOUR-REGION>-<PROJECT-ID>.cloudfunctions.net/runAgentStep";

  try {
    const response = await fetch(apiEndpoint, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        prompt: document.getElementById("user-prompt").value,
        domTree: domData
      })
    });

    const data = await response.json();
    console.log("Agent Decision Matrix:", JSON.parse(data.response));

    // Pass execution payload back to extension background/content script for automation
  } catch (err) {
    console.error("Failed to communicate with agent backend:", err);
  }
});
```

Step 4: Structuring GitHub and Automated CI/CD Deployment

Managing your repository professionally ensures smooth feature iteration. Initialize your local Git project:

```bash
git init
git add .
git commit -m "chore: initial autonomous agent scaffold"
git branch -M main
git remote add origin https://github.com/yourusername/ai-agent-chrome-extension.git
git push -u origin main
```

Automating Deployments with GitHub Actions

Create a workflow configuration file at `.github/workflows/firebase-deploy.yml`. This automatically provisions and deploys updates to your backend functions whenever code hits the `main` branch.

```yaml
name: Deploy Backend to Firebase

on:
** push:**
**   branches:**
**   - main**

jobs:
** deploy_firebase:**
**   runs-on: ubuntu-latest**
**   steps:**
**   - name: Checkout Repository**
**   uses: actions/checkout@v4**

**   - name: Setup Node.js Environment**
**   uses: actions/setup-node@v4**
**   with:**
**   node-version: 20**

Example with Bash and Python

Prerequisites

Install the required packages and Playwright browsers:

Bash

```
None
pip install google-genai playwright
playwright install
```

Set your API key in your environment:

Bash

```
None
export GEMINI_API_KEY="your_gemini_api_key_here"
```

Python Script: `agent.py`

Python

```
None
import os
import json
import asyncio
from google import genai
from google.genai import types
from playwright.async_api import async_playwright

# Initialize the Gemini Client
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

SYSTEM_INSTRUCTION = """
You are an autonomous web automation agent.
Given a JSON payload describing interactive DOM elements on the
current page and a user goal, return a JSON object containing:
- "thought": Your step-by-step reasoning for this action.
- "action": The action to take, which must be one of:
    - {"type": "CLICK", "selector": "CSS selector"}
    - {"type": "TYPE", "selector": "CSS selector", "text": "text to
type"}
    - {"type": "DONE", "message": "Summary of completed task"}
Return strictly valid JSON only. Do not include markdown code block
formatting.
"""

async def extract_page_context(page):
    """Scrapes visible, interactive elements from the active web
page."""
    return await page.evaluate("""
        () => {
            const elements =
Array.from(document.querySelectorAll('button, a, input, textarea,
select'));
            return elements.map(el => ({
                tag: el.tagName.toLowerCase(),
                id: el.id || null,
                class: el.className || null,
                name: el.getAttribute('name') || null,
```

```
                type: el.getAttribute('type') || null,
                text: el.innerText ? el.innerText.trim() :
(el.placeholder || ''),
                selector: el.id ? `#${el.id}` : (el.name ?
`[name="${el.name}"]` : el.tagName.toLowerCase())
            })).filter(el => el.selector !== '');
        }
    """)

async def run_agent(task: str, start_url: str):
    async with async_playwright() as p:
        # Launch browser in visible mode
        browser = await p.chromium.launch(headless=False)
        page = await browser.new_page()
        await page.goto(start_url)

        step = 1
        max_steps = 10

        print(f"🚀 Starting task: '{task}' at {start_url}\n")

        while step <= max_steps:
            print(f"--- Step {step} ---")
            await page.wait_for_load_state("networkidle")

            # 1. Capture DOM context
            dom_elements = await extract_page_context(page)

            prompt = f"User Goal: {task}\nAvailable Page Elements:
{json.dumps(dom_elements, indent=2)}"

            # 2. Call Gemini API for decision
            response = client.models.generate_content(
                model="gemini-2.5-flash",
                contents=prompt,
                config=types.GenerateContentConfig(
                    system_instruction=SYSTEM_INSTRUCTION,
                    response_mime_type="application/json"
                )
            )
```

```
            decision = json.loads(response.text)
            print(f"Thinking: {decision.get('thought')}")

            action = decision.get("action", {})
            action_type = action.get("type")

            # 3. Execute Decision
            if action_type == "CLICK":
                print(f"Executing: CLICK on
{action.get('selector')}")
                await page.click(action.get("selector"))

            elif action_type == "TYPE":
                print(f"Executing: TYPE '{action.get('text')}' into
{action.get('selector')}")
                await page.fill(action.get("selector"),
action.get("text"))
                await page.keyboard.press("Enter")

            elif action_type == "DONE":
                print(f"\n✅ Task Completed:
{action.get('message')}")
                break

            else:
                print(f"Unknown action: {action}")
                break

            step += 1
            await asyncio.sleep(2)  # Pause briefly for page
transition

        await browser.close()

if __name__ == "__main__":
    # Example usage:
    asyncio.run(run_agent(
        task="Search for Python web scraping techniques",
        start_url="https://www.google.com"
```

```
    ))
```

How It Works

1. **DOM Extraction: Uses Playwright's `evaluate` function to extract visible interactive elements (`button`, `input`, `a`) along with generated CSS selectors.**
2. **Structured Prompting: Sends the simplified DOM representation and user task to `gemini-2.5-flash`.**
3. **JSON Response Enforcement: Uses `response_mime_type="application/json"` to guarantee machine-readable output from Gemini without raw markdown code fences.**
4. **Browser Actions: Parses the decision payload and issues async commands (`page.click()`, `page.fill()`) back to Playwright.**

See more examples and learn more at https://www.amazinggracehl.org/lms/
Nicholai.Online